université
PARIS-SACLAY

Master SETI
2026-2027

RAPPORT DE TP

TP 1 : Calcul de racine carrée

Côme CERVANTES Joseph MOUSCADET

Nous attestons que ce travail est original,
qu'il est le fruit d'un travail commun au binôme et qu'il a été rédigé de manière autonome.

Paris, le 07/10/2026

I. Architecture séquentielle comportementale

A. Principe et graphe d'état

Question a). Nous cherchons à calculer la racine carrée entière d'un nombre non signé A codé sur $2n$ bits, avec un résultat sur n bits. La description doit être synthétisable, utiliser un seul processus et réaliser chacune des n itérations en un cycle. Le résultat attendu est $Z = \lfloor \sqrt{A} \rfloor$, ce qui se vérifie par $Z^2 \leq A < (Z+1)^2$.

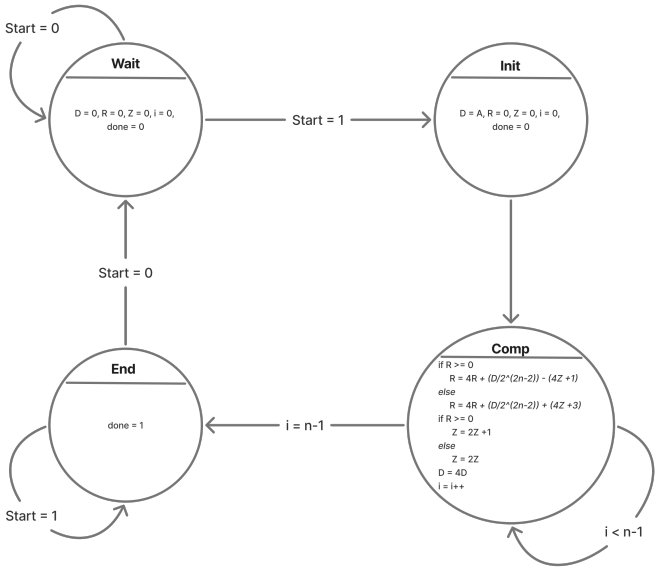


Figure 1 – Graphe d'état utilisé pour organiser l'architecture séquentielle.

Le graphe de la figure 1 sépare l'attente, le chargement, le calcul et la fin du traitement. Il correspond aux quatre états du type énuméré utilisé dans `sqrt_seq.vhd` :

- `wait_state` : mise à zéro de D, R, Z, du compteur et de `done`. Le circuit attend `start = '1'` pour passer à l'initialisation.
- `init_state` : chargement de A dans D, remise à zéro des autres données, puis passage au calcul au cycle suivant.
- `comp_state` : exécution d'une itération par front montant. L'état est conservé tant que $i < n-1$; la dernière itération conduit à `end_state`.
- `end_state` : `done` passe à 1. Le résultat est conservé ; le circuit reste dans cet état si `start = '1'`, sinon il retourne à l'attente.

Les actions des états sont exécutées aux fronts montants de l'horloge. Le reset actif à 1 impose l'état d'attente de manière asynchrone ; les données sont ensuite initialisées au front exécutant cet état. Le compteur part de $i = 0$ et parcourt les valeurs de 0 à $n-1$, soit exactement n itérations. La boucle de calcul est empruntée si $i < n-1$; si $i = n-1$, le circuit effectue la dernière itération puis passe à l'état de fin. Les conditions portent sur la valeur du compteur avant sa mise à jour. Pour $n = 32$, les indices vont donc de 0 à 31.
Dans le graphe, la division $D/2^{2n-2}$ désigne une division entière, réalisée dans le code en sélectionnant les deux bits de poids fort de D. L'incrément indiqué dans l'état de calcul s'applique aux itérations non finales : le code effectue $i \leftarrow i+1$ lorsque $i < n-1$ et remet i à zéro à la dernière itération. Le premier test de signe utilise l'ancien reste R ; le second utilise le reste qui vient d'être calculé. Cette distinction permet de produire un bit de racine à chaque cycle.

B. Algorithme et choix des types

L'algorithme construit la racine bit par bit, en traitant les bits de A par paires, depuis les poids forts. Au début d'une itération, on extrait $p = D(2n-1 \text{ downto } 2n-2)$, soit un entier entre 0 et 3. Avec les anciennes valeurs de R et Z, on calcule :

$$R_{\text{nouveau}} = \begin{cases} 4R + p - (4Z+1), & R \geq 0, \\ 4R + p + (4Z+3), & R < 0. \end{cases}$$

Le signe du *nouveau* reste détermine ensuite le bit ajouté à la racine :

$$Z_{\text{nouveau}} = \begin{cases} 2Z+1, & R_{\text{nouveau}} \geq 0, \\ 2Z, & R_{\text{nouveau}} < 0. \end{cases} \qquad D_{\text{nouveau}} = (4D) \bmod 2^{2n}.$$

Les multiplications par 2 et par 4 sont réalisées par des décalages à gauche de un ou deux bits. L'extraction des deux bits de poids fort remplace la division entière par 2^{2n-2}. Le décalage de D élimine la paire déjà traitée et présente la suivante. Il n'est donc pas nécessaire d'utiliser un diviseur ou un multiplicateur général.

Élément	**Type et taille pour $n = 32$**	**Rôle et justification**
`clk, rst, start, done`	`std_logic`, 1 bit	Signaux de commande et indication de fin.
`A`	`std_logic_vector`, 64 bits	Bus d'entrée ; converti en `unsigned` lors du chargement.
`Result`	`std_logic_vector`, 32 bits	Bus de sortie, relié à Z par une conversion de type.
D (signal)	`unsigned`, $2n = 64$ bits	Copie décalée de l'opérande, toujours non signée.
Z (signal)	`unsigned`, $n = 32$ bits	Racine partielle puis résultat entier non négatif.
R (variable)	`signed`, $n + 2 = 34$ bits	Reste partiel pouvant être négatif ; les bits supplémentaires servent au signe et aux opérations intermédiaires.
i (signal)	`integer range 0 to n-1`	Compteur borné de 0 à 31, correspondant aux 32 itérations.
`curr_state`	Type énuméré à quatre valeurs	Rend explicites les phases de commande de l'automate.

Table 1 – Signaux et variable de l'architecture comportementale.

Le paramètre générique n fixe la largeur du résultat et le nombre d'itérations ; $n = 32$ donne une entrée sur 64 bits. Le paquet `numeric_std` fournit les opérations sur `signed` et `unsigned`. Les conversions et les appels à `resize` mettent les opérandes à la largeur de R avant les additions, soustractions et décalages. Contrairement à la boucle décroissante du sujet, notre compteur est croissant : l'ordre des paires traitées reste identique, car il est imposé par le décalage de D.

C. Un seul processus et une itération par cycle

La commande et le calcul sont regroupés dans le processus `PCalc`, sensible à `clk` et `rst`. Hors reset, les mises à jour se font sur les fronts montants. L'affectation concurrente de `Result` à partir de Z ne constitue pas un second processus de commande ; elle expose simplement le registre de résultat.

Pourquoi une variable pour R ? Une affectation de variable avec `:=` prend effet immédiatement dans l'exécution du processus. Le second test de signe utilise ainsi le reste qui vient d'être calculé au même front d'horloge. Une affectation de signal avec `<=`, suivie du même test dans ce processus, lirait encore l'ancienne valeur. Ce choix permet de respecter l'enchaînement de l'algorithme en un seul cycle par itération.

La variable R conserve néanmoins sa valeur entre deux activations du processus : elle représente ici une donnée mémorisée, et non une valeur automatiquement remise à zéro à chaque cycle. Les signaux D, Z et i sont également mémorisés, mais leurs nouvelles valeurs ne sont visibles qu'après l'exécution du processus. Les calculs du cycle utilisent donc l'ancien Z et l'ancien D, conformément aux équations précédentes.

Interface de commande. L'entrée A doit être stable au front exécutant `init_state`, où elle est effectivement copiée dans D. Elle n'est plus utilisée pendant les itérations. La sortie `Result` présente des valeurs partielles pendant le calcul ; elle doit être exploitée lorsque `done = '1'`. Maintenir `start` à 1 permet de conserver l'état de fin. Après sa remise à 0, l'automate retourne à l'attente, puis efface le résultat lors de l'exécution de cet état.

D. Temps de calcul et organisation du testbench

Le cœur du calcul réalise exactement n itérations, donc n cycles. Le protocole ajoute un cycle d'initialisation et un cycle pour exécuter l'état de fin et lever `done`. À partir du front qui détecte `start` dans l'état d'attente, les délais sont donc :

$$N_{\text{Result final}} = n + 1, \qquad N_{\text{done}} = n + 2, \qquad t_{\text{done}} = (n + 2)T_{\text{clk}}.$$

Ces nombres désignent des *intervalles* d'horloge depuis le front de détection. Depuis le chargement de A, `done` arrive après $n + 1$ cycles. Le nombre d'itérations reste constant, quelle que soit la valeur de A.

Dans `testbench.vhd`, $n = 32$, le reset dure 5 ns et l'horloge a une période de 20 ns, soit 50 MHz en simulation. Le premier `start` est activé à 25 ns, détecté au front de 30 ns. Le déroulement déduit du code est :

Événement du premier calcul	**Instant**
Détection de `start`	30 ns
Chargement de $A = 100$ dans `init_state`	50 ns
Première / dernière itération	70 ns / 690 ns
Résultat final / activation de `done`	690 ns / 710 ns

Table 2 – Chronologie attendue d'après le testbench et l'automate.

Le calcul itératif représente 640 ns ; le délai entre la détection de `start` et `done` vaut 680 ns. Le second opérande, 25, est présenté à 785 ns, avec un nouveau `start` à 805 ns : sa détection est attendue à 810 ns et `done` à 1490 ns. Ces durées utilisent l'horloge du testbench ; elles ne constituent pas une mesure de fréquence maximale sur FPGA.

Vérification du temps de calcul en simulation

Mesures relevées en simulation. Pour $n = 32$ et $T_{\text{clk}} = 20$ ns, les instants relevés sur les chronogrammes sont regroupés ci-dessous. Les valeurs initialement exprimées en ps sont converties en ns (1 000 ps = 1 ns). Le départ mesuré correspond à la montée de `start`, et non au front d'horloge qui accepte la demande.

Entrée A	Montée de start	Montée de done	Délai mesuré
1 194 877 489	25 ns	710 ns	685 ns
0	25 ns	710 ns	685 ns
1 (après 0)	805 ns	1490 ns	685 ns

Table 3 – Mesures temporelles relevées dans ModelSim pour trois calculs, avec une période de 20 ns.

Dans les deux premiers essais, `start` monte à 25 ns et est accepté au front montant de 30 ns. Dans le troisième, il monte à 805 ns et est accepté à 810 ns. Dans les trois cas, on obtient donc :

$$t_{\text{start}\uparrow \to \text{done}\uparrow} = 5 \text{ ns} + 34 \times 20 \text{ ns} = 685 \text{ ns},$$
$$t_{\text{acceptation} \to \text{done}\uparrow} = (n+2)T_{\text{clk}} = 680 \text{ ns}.$$

Les 5 ns supplémentaires correspondent à l'attente du prochain front montant. Le cœur itératif nécessite $nT_{\text{clk}} = 32 \times 20 = 640$ ns ; cette durée est déduite des 32 cycles de calcul. Les mesures de 685 ns comprennent également cette attente et les cycles du protocole. Elles confirment la latence prévue par le code pour les trois opérandes relevés. Le même délai a été observé pour les autres calculs réalisés ; l'architecture effectue toujours 32 itérations, quelle que soit la valeur de A. Ces observations valident le temps en simulation, sans mesurer la fréquence maximale du circuit sur FPGA.

E. Résultats des simulations

Les captures fournies montrent quatre couples d'opérandes. Le tableau 4 compare les racines entières attendues aux résultats lisibles sur les chronogrammes. Les carrés parfaits, les petites valeurs et les valeurs non carrées permettent de vérifier plusieurs cas de fonctionnement.

Entrée A	$\lfloor\sqrt{A}\rfloor$ attendu	Résultat observé	Bilan
0	0	0	Conforme
1	1	1	Conforme
15	3	3	Conforme
25	5	5	Conforme
100	10	10	Conforme
512	22	22	Conforme
1 194 877 489	34 567	34 567 au curseur	Conforme
4 294 967 295	65 535	65 535 au curseur	Conforme

Table 4 – Comparaison entre résultats attendus et lecture des captures.

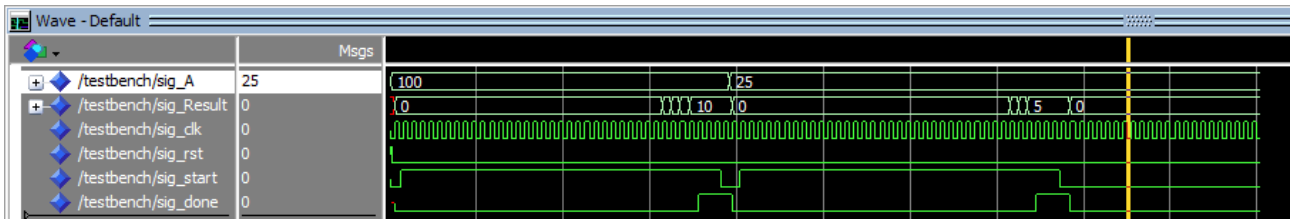


Figure 2 – Essais $A = 100$ puis $A = 25$: résultats 10 et 5.

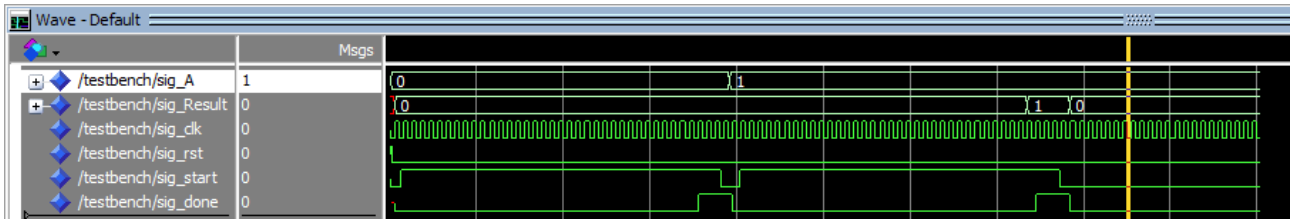


Figure 3 – Essais $A = 0$ puis $A = 1$: résultats 0 et 1.

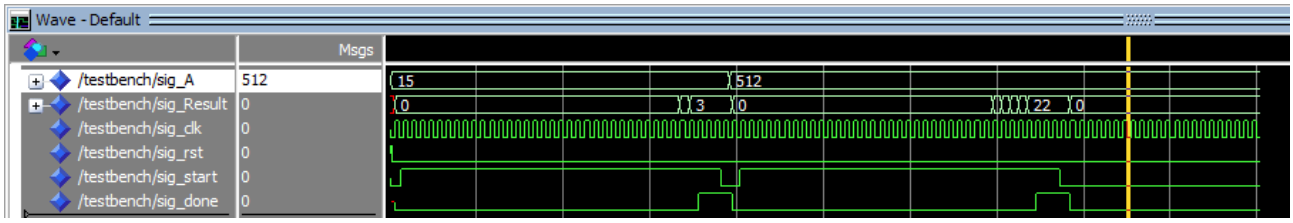


Figure 4 – Essais $A = 15$ puis $A = 512$: résultats entiers 3 et 22.

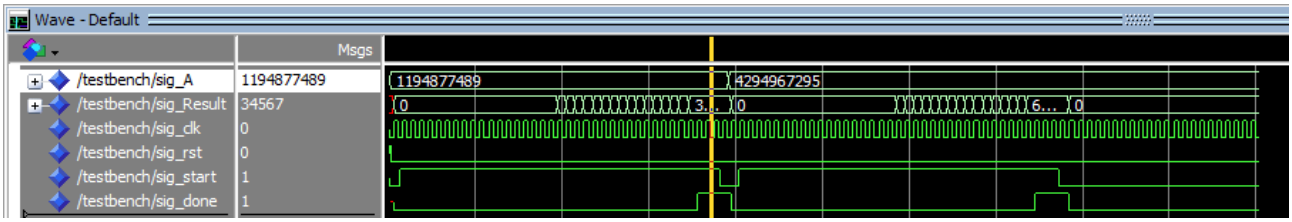


Figure 5 – Confirmation du premier grand opérande au curseur : $A = 1\,194\,877\,489$, `Result=34567` et `done=1`.

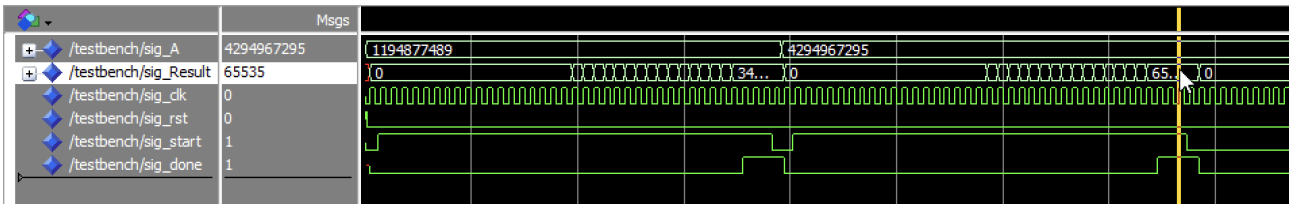


Figure 6 – Essais sur de grandes valeurs ; au curseur, $A = 4\,294\,967\,295$, `Result=65535` et `done=1`.

F. Bilan de la question a)

La description comportementale de `sqrt_seq.vhd` regroupe la commande et le calcul dans un seul processus horlogé, `PCalc`. Les opérations portent sur des données de largeur fixe et les décalages sont constants ; aucun délai de simulation n'est utilisé dans le circuit. Chaque passage dans `comp_state` réalise une itération complète de l'algorithme, conformément à la contrainte du sujet.

La simulation avec $n = 32$ utilise une entrée de 64 bits et un résultat de 32 bits. Les huit résultats documentés sont conformes à la racine carrée entière attendue. Les essais comprennent des carrés parfaits, les cas 0 et 1, ainsi que des nombres non carrés : $3^2 \leq 15 < 4^2$, $22^2 \leq 512 < 23^2$ et $65\,535^2 \leq 4\,294\,967\,295 < 65\,536^2$. Les figures 5 et 6 apportent les lectures au curseur des deux grands opérandes : respectivement 34 567 et 65 535, avec `done=1`. Le résultat est lu lorsque `done=1`. Pour construire les grands opérandes dans le banc de test sans dépasser la capacité du type entier de l'outil, un littéral hexadécimal de 64 bits peut être utilisé, par exemple `x"00000000FFFFFFFF"`.

Le temps de calcul demandé par le sujet est de n cycles pour les n itérations. Avec le protocole de cette implémentation, le résultat final est disponible après $n + 1$ périodes depuis l'acceptation de `start`, et `done` est levé après $n + 2$ périodes. Pour $n = 32$ et $T_{\text{clk}} = 20$ ns, cela représente 640 ns de calcul et 680 ns jusqu'à `done`. Les mesures de 685 ns depuis la montée de `start` incluent 5 ns d'attente du front d'acceptation et confirment cette estimation.

Cette validation porte sur les valeurs simulées : le banc présent dans le dossier contient les essais 100 et 25, les autres étant documentés par les captures et les relevés de simulation. La fréquence maximale et les ressources logiques seront à mesurer lors de l'implémentation sur carte pour la question c).

II. Architecture séquentielle structurelle

A. Organisation de l'architecture

Question b). Le sujet demande une description structurelle comportant un chemin de données à un seul additionneur et une unité de contrôle, avec des blocs définis dans des entités séparées. Le modèle doit être validé en simulation pour $n = 32$, puis son temps de calcul doit être exprimé en nombre de cycles en fonction de n.
Nous reprenons la démarche de la partie *Impose a specific architecture* du cours sur le multiplicateur : définir les registres, multiplexeurs et additionneur, concevoir leur commande, puis assembler les blocs par instanciation. L'entité principale `sqrt_struct`, d'architecture `seq_det`, instancie l'unité de contrôle `CU` et le chemin de données `DP`. Celui-ci instancie trois registres, trois multiplexeurs et un additionneur. Les décalages constants sont réalisés par câblage à l'entrée des registres.

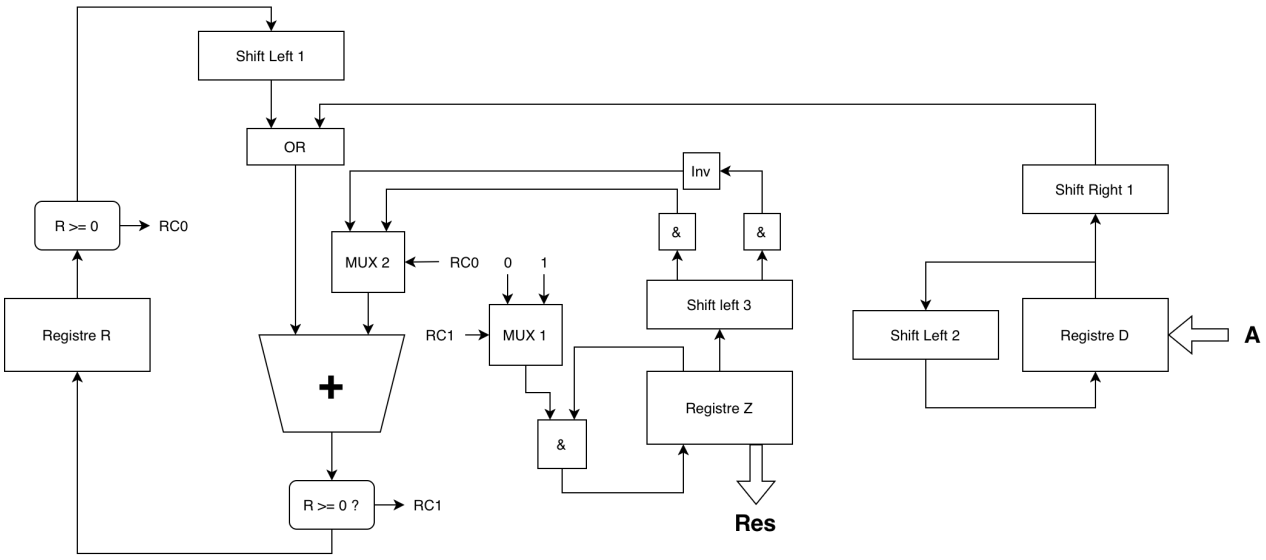


Figure 7 – Schéma de principe du chemin de données proposé pour la question b). Les commandes de chargement et d'initialisation sont détaillées dans le texte.

Les libellés *Shift Left 1, 2, 3* identifient les blocs : les branches qui forment $4R$, $4D$ et $4Z$ décalent chacune de **deux bits**. Le bloc d'extraction sur D fournit les deux bits de poids fort, soit $p = \lfloor D/2^{2n-2} \rfloor$. Les symboles `&` désignent des concaténations de bits. Le premier test de signe porte sur le registre R, le second sur la sortie de l'additionneur, c'est-à-dire le nouveau reste. Ces tests ne coupent pas les bus de données : ils en prélèvent seulement le bit de signe.

Entité VHDL	Fonction
`sqrt_struct`	Assemblage de CU et DP ; interface clk, reset, start, A, S, done.
`unite_controle`	Automate, compteur d'itérations et commandes init/calcul.
`chemin_donnees`	Connexions des blocs et formation des valeurs suivantes de R, D et Z.
`registre`	Mémoire avec reset asynchrone, remise à zéro synchrone et chargement.
`mux2_1`	Sélection combinatoire entre deux entrées.
`additionneur`	Addition à largeur fixe ; une seule instance dans DP.

Table 5 – Décomposition de l'architecture en entités séparées.

B. Réaliser l'itération avec un seul additionneur

On conserve les registres D sur $2n$ bits, Z sur n bits et R sur $W = n + 2$ bits. Pour $n = 32$, cela donne respectivement 64, 32 et 34 bits. Avec p égal à la paire supérieure de D, les deux opérandes de l'additionneur sont :

$$X = (4R)\,\mathrm{OR}\,p, \qquad Y = \begin{cases} \mathrm{NOT}(4Z), & R \geq 0, \\ 4Z + 3, & R < 0. \end{cases} \qquad R_{\text{suivant}} = X + Y.$$

Les deux bits bas de $4R$ sont nuls et p n'occupe que ces deux bits : le OR réalise donc ici $4R + p$ sans additionneur supplémentaire. Les opérandes sont ramenés à W bits.

Les concaténations $Z \,\&\, \texttt{00}$ et $Z \,\&\, \texttt{11}$ donnent respectivement $4Z$ et $4Z + 3$. En complément à deux, $\mathrm{NOT}(x) = -x - 1$ modulo 2^W ; l'inversion de $4Z$ produit donc $-(4Z + 1)$. Le multiplexeur `MUX2`, commandé par le signe de l'ancien reste, choisit Y. L'unique instance `ADD` calcule alors le nouveau reste.

Le signe de ce nouveau reste sélectionne, via `MUX1`, le bit b ajouté à la racine : $b = 1$ si $R_{\text{suivant}} \geq 0$, et $b = 0$ sinon. La valeur suivante de Z est obtenue en concaténant les $n - 1$ bits bas de Z avec b, soit $2Z + b$. De même, D est décalé de deux bits pour présenter la paire suivante. Un troisième multiplexeur, `MUX_D`, sélectionne A à l'initialisation ou D décalé pendant le calcul ; il complète le schéma de principe.

Les trois registres mémorisent leurs nouvelles valeurs au même front de calcul. Les décalages ne nécessitent donc aucun état supplémentaire. Le décrément du compteur appartient à l'unité de contrôle ; la contrainte d'un seul additionneur concerne le chemin de données. L'addition sur W bits est modulaire : après k itérations, $0 \leq Z \leq 2^k - 1$ et $-(2Z + 1) \leq R \leq 2Z$, ce qui permet de représenter le reste dans $n + 2$ bits.

C. Unité de contrôle suivant le modèle du cours

Comme dans l'exemple du cours, la commande sépare la mémorisation de l'état (`CSM`), le calcul de l'état suivant (`NSC`) et le calcul des sorties (`OC`). Les sorties `init`, `calcul` et `done` sont combinatoires à partir de l'état : elles changent après la transition d'état et sont disponibles avant le prochain front de chargement des registres.

État	init	calcul	done	Action et transition
`wait_state`	0	0	0	Conserver les données ; attendre start=1.
`init_state`	1	0	0	Au front suivant : D=A, R=Z=0, $i = n - 1$; entrer en calcul.
`comp_state`	0	1	0	Une itération par front ; décrémenter si i>0 ; si i=0, réaliser la dernière itération et entrer en fin.
`end_state`	0	0	1	Conserver S ; attendre start=0 pour revenir en attente.

Table 6 – États et commandes de l'unité de contrôle de QB.

Le reset remet l'automate en attente et annule les données des registres. Les signes RC0 et RC1 commandent directement les multiplexeurs de DP ; ils n'interviennent pas dans les transitions d'état. L'organisation du cours est adaptée à la racine carrée : les états de décalage du multiplicateur ne sont pas nécessaires ici.

D. Estimation du temps de calcul

Soit F0 le front montant qui accepte `start` dans l'état d'attente. L'automate entre en initialisation. Au front F1, A est capturé et les registres sont initialisés. La première itération est réalisée à F2 et la dernière à F($n+1$). À ce dernier front, l'automate entre dans `end_state` et `done` monte après stabilisation des signaux combinatoires.

$$N_{\text{itérations}} = n, \qquad N_{\text{acceptation}\to\text{done}} = n+1, \qquad t_{\text{acceptation}\to\text{done}} = (n+1)T_{\text{clk}}.$$

Pour $n = 32$, le calcul comporte donc 32 cycles et la latence jusqu'à `done` est de 33 périodes depuis F0. Le banc QB utilise une période de 10 ns : les durées théoriques sont 320 ns pour le calcul et 330 ns depuis l'acceptation jusqu'à `done`. Une mesure depuis la montée de `start` inclut en plus l'attente du front F0. Les mesures présentées ci-après permettent de vérifier cette estimation.

Dans QA, `done` est affecté au front exécutant l'état final ; dans QB, il est décodé dès l'entrée dans cet état. La latence du protocole est ainsi réduite d'une période. Cette comparaison en cycles ne suffit pas à conclure sur les performances sur carte : les fréquences maximales et les ressources relèvent de la question c).

E. Validation en simulation avec n=32

La simulation est réalisée dans ModelSim avec le banc `tb_sqrt_struct.vhd`, une entrée sur 64 bits et une sortie sur 32 bits. Le script `simuler_modelsim.do` compile les blocs en VHDL-93, charge le banc et affiche les signaux. Le banc comporte 17 calculs vérifiés par assertions : petites valeurs, carrés et voisins, opérandes dépassant 32 bits et valeur maximale sur 64 bits. Il contrôle aussi les 32 itérations par calcul, la capture de l'entrée, le maintien du résultat, le reset en cours d'opération et le redémarrage.

```
# ** Warning: (vsim-WLF-5000) WLF file currently in use: vsim.wlf
#           File in use by: Côme  Hostname: WIN-NIFFGN3OTFM  ProcessID: 9912
#           Attempting to use alternate WLF file "./wlftb0ec2q".
# ** Warning: (vsim-WLF-5001) Could not open WLF file: vsim.wlf
#           Using alternate file: ./wlftb0ec2q
# ** Note: PASS : 17 calculs, 32 iterations par calcul, reset et start/done
#    Time: 6346 ns  Iteration: 0  Instance: /tb_sqrt_struct
# 0 ps
# 6667500 ps
```

Figure 8 – Fin du banc ModelSim : message PASS à 6 346 ns pour les 17 calculs et les contrôles du protocole.

La figure 8 montre que le banc atteint son message de réussite. Les avertissements visibles concernent uniquement un fichier de chronogrammes WLF déjà utilisé : ModelSim annonce l'emploi d'un fichier alternatif. L'instant de 6 346 ns correspond à la fin de la campagne de tests, et non à la durée d'un calcul.

F. Résultats observés sur les chronogrammes

Les captures associent, pour chaque essai, la valeur d'entrée lors de la montée de `start` et le résultat à la montée de `done`. Le banc inverse volontairement A après sa capture pour vérifier que le circuit utilise la donnée mémorisée : la grande valeur visible sur A à la fin ne constitue donc pas un nouvel opérande pour le calcul en cours. De même, la valeur de S au départ peut être le résultat de l'essai précédent ; seule sa lecture lorsque `done=1` valide le résultat courant.

Entrée capturée A	Résultat attendu	S avec done=1	Bilan
0	0	0	Conforme
15	3	3	Conforme
100	10	10	Conforme
4 294 967 296	65 536	65 536	Conforme

Table 7 – Comparaison des résultats attendus et observés pour quatre opérandes représentatifs.

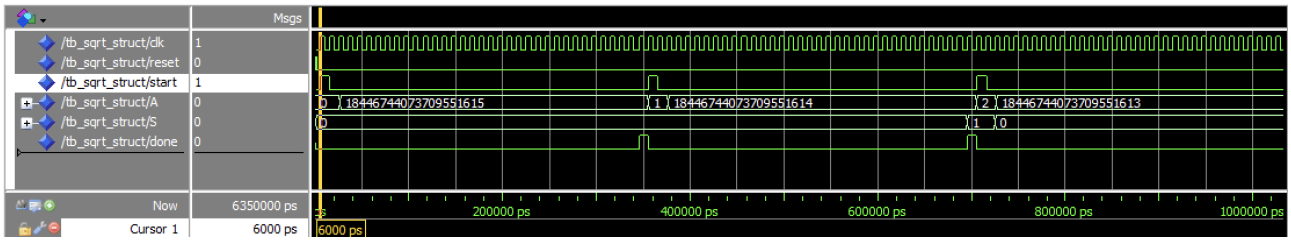


Figure 9 – Essai $A = 0$: entrée et start relevés à 6 000 ps.

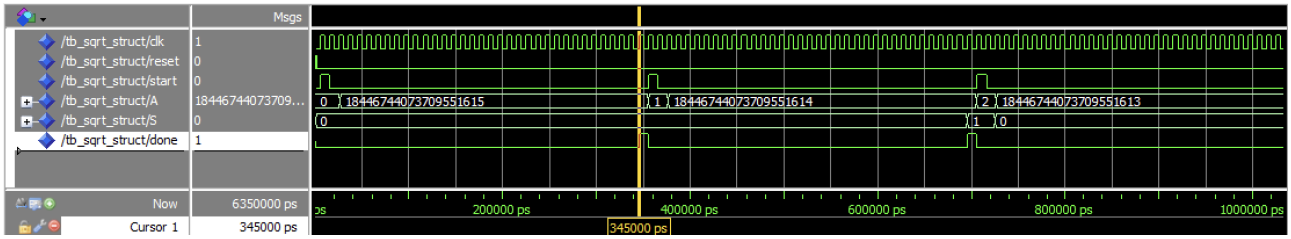


Figure 10 – Essai $A = 0$: $S = 0$ et done=1 à 345 000 ps.

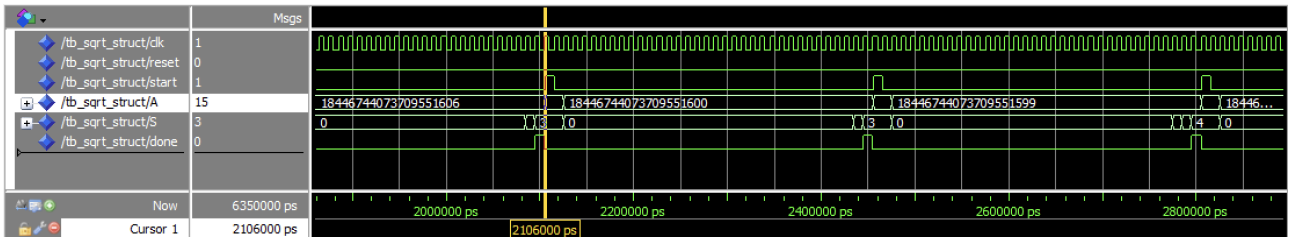


Figure 11 – Essai $A = 15$: entrée et start relevés à 2 106 000 ps.

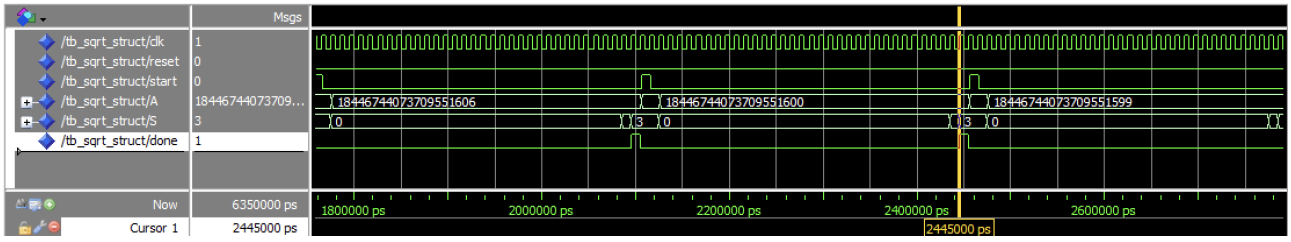


Figure 12 – Essai $A = 15$: $S = 3$ et done=1 à 2 445 000 ps.

Carré parfait et opérande supérieur à 32 bits

L'essai $A = 100$ vérifie un carré parfait : $10^2 = 100$. L'essai $A = 4\,294\,967\,296 = 2^{32}$ met en jeu un opérande qui nécessite plus de 32 bits ; le résultat est $65\,536 = 2^{16}$. Avec $A = 15$, pour lequel $3^2 \leq 15 < 4^2$, ces cas illustrent le calcul de la racine carrée entière sur des valeurs de natures différentes.

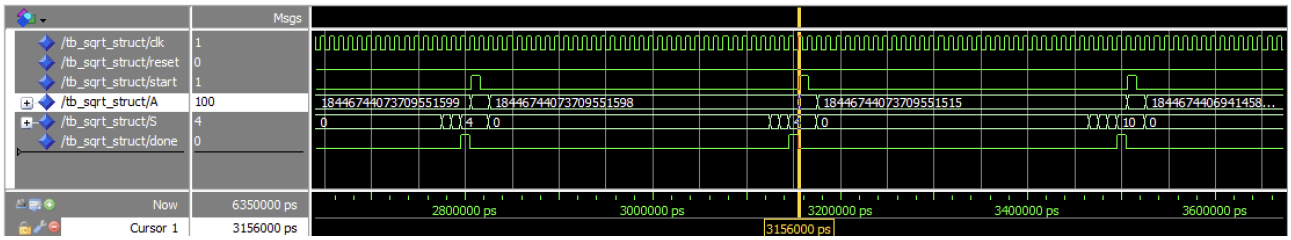


Figure 13 – Essai $A = 100$: entrée et start relevés à 3 156 000 ps.

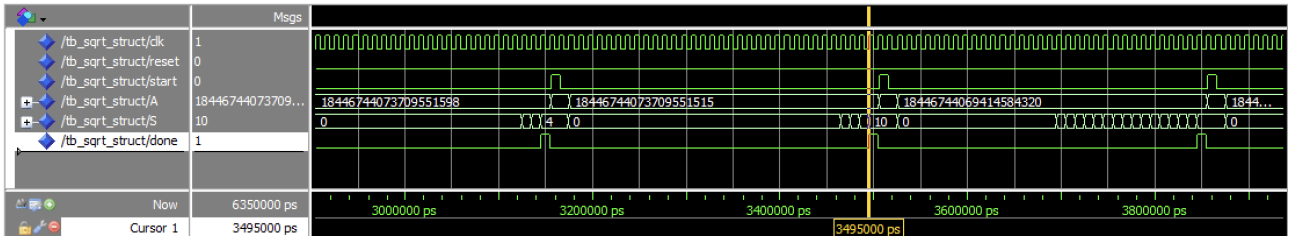


Figure 14 – Essai $A = 100$: $S = 10$ et done=1 à 3 495 000 ps.

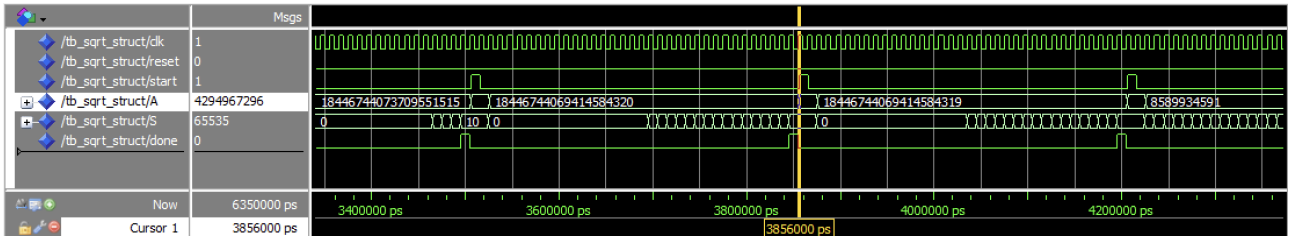


Figure 15 – Essai $A = 4\,294\,967\,296$: entrée et start relevés à 3 856 000 ps.

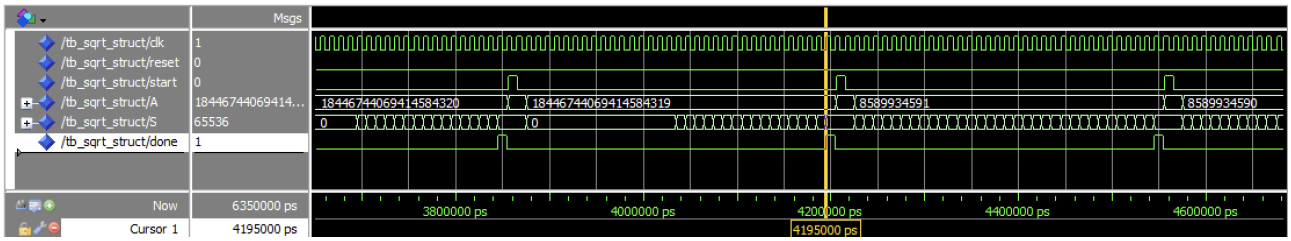


Figure 16 – Essai $A = 4\,294\,967\,296$: $S = 65\,536$ et done=1 à 4 195 000 ps.

G. Vérification du compteur et du temps de calcul

La figure 17 détaille la fin d'une opération et le lancement de l'essai $A = 9$. En calcul, le compteur descend jusqu'à zéro. L'itération correspondant à $i = 0$ est exécutée avant l'entrée dans `end_state`, où `done` monte. Le retour passe ensuite par `wait_state`, puis par `init_state`. Au front d'initialisation du calcul suivant, i est rechargé à 31, puis décrémenté à chaque itération.

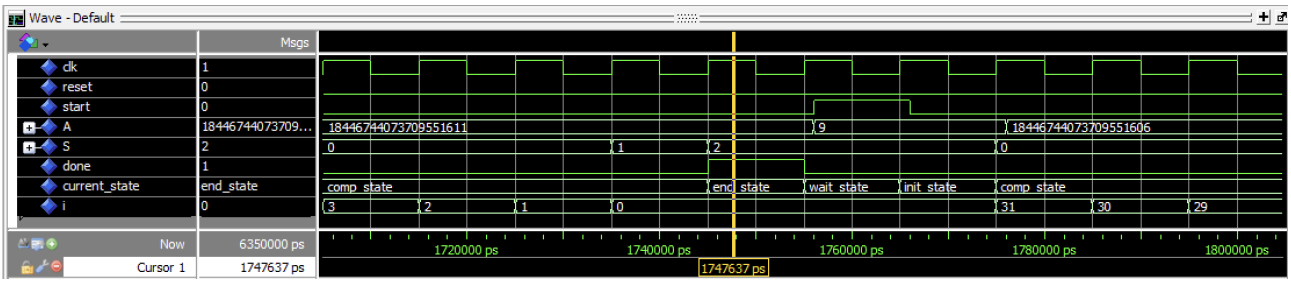


Figure 17 – Détail des transitions et du compteur : fin de calcul avec $i = 3, 2, 1, 0$, puis initialisation du calcul de $A = 9$ avec $i = 31$.

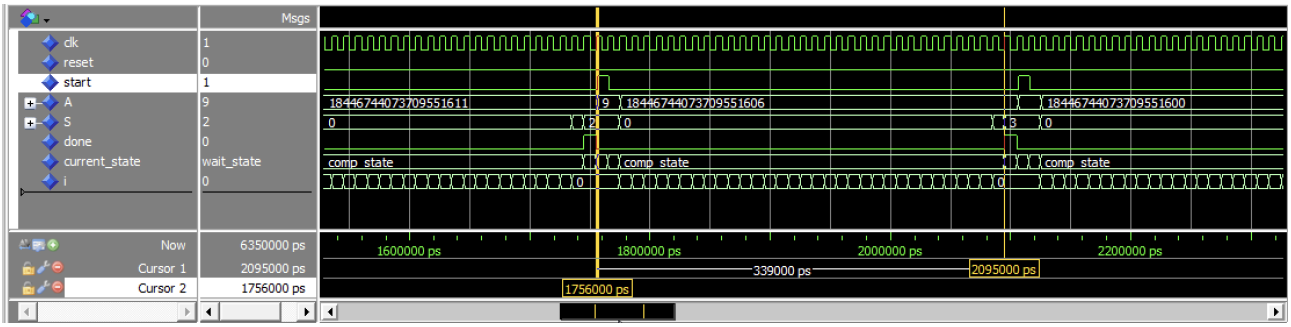


Figure 18 – Calcul de $A = 9$: mesure entre la montée de start à 1 756 000 ps et celle de done à 2 095 000 ps ; $S = 3$ en fin de calcul.

Pour cet essai, start monte à 1 756 ns et est accepté au front suivant à 1 765 ns. L'initialisation est réalisée à 1 775 ns et done monte à 2 095 ns. Le délai de 339 000 ps mesuré avec les deux curseurs se décompose ainsi :

$$t_{\text{start}\uparrow\rightarrow\text{done}\uparrow} = 2\,095 - 1\,756 = 339 \text{ ns},$$
$$t_{\text{acceptation}\rightarrow\text{done}\uparrow} = 2\,095 - 1\,765 = 330 \text{ ns} = 33T_{\text{clk}},$$
$$t_{\text{calcul}} = 2\,095 - 1\,775 = 320 \text{ ns} = 32T_{\text{clk}}.$$

Les 9 ns supplémentaires entre la montée de start et son acceptation proviennent de la position du stimulus entre deux fronts d'horloge. Elles ne constituent pas une itération supplémentaire.

H. Synthèse des mesures et bilan de la question b)

Les quatre autres couples de captures donnent le même délai de 339 ns entre start et done. Le tableau 8 regroupe les instants relevés, convertis en ns (1 000 ps = 1 ns), et les fronts d'acceptation déduits de l'horloge de période 10 ns.

A	**start monte**	**Acceptation**	**done monte**	**Délai start/done**
0	6 ns	15 ns	345 ns	339 ns
9	1 756 ns	1 765 ns	2 095 ns	339 ns
15	2 106 ns	2 115 ns	2 445 ns	339 ns
100	3 156 ns	3 165 ns	3 495 ns	339 ns
4 294 967 296	3 856 ns	3 865 ns	4 195 ns	339 ns

Table 8 – Instants relevés dans ModelSim et fronts d'acceptation correspondants pour QB.

Dans tous les cas, le délai depuis le front d'acceptation est de 330 ns, soit $n + 1 = 33$ périodes. Les $n = 32$ itérations occupent 320 ns. La durée est indépendante de l'opérande : le circuit traite systématiquement les 32 paires de bits, y compris pour les petites valeurs.

L'architecture répond à la décomposition structurelle demandée : le sommet instancie CU et DP, et le chemin de données utilise des entités distinctes pour les registres, les multiplexeurs et l'unique additionneur. La transformation des équations en concaténations, inversion et sélection permet d'effectuer une itération complète par cycle avec cette unique instance d'addition. L'unité de contrôle suit la séparation CSM/NSC/OC du cours et fournit les commandes combinatoires aux registres.

La validation ModelSim avec $n = 32$ est confirmée par les résultats au curseur, l'évolution du compteur et le message de réussite des 17 calculs. Les contrôles de reset, de maintien de start et de redémarrage sont réalisés par les assertions du banc ; ils ne font pas ici l'objet de captures spécifiques. Les mesures sont conformes à l'estimation générale : n cycles de calcul et $n + 1$ périodes jusqu'à done depuis l'acceptation de start.

Par rapport à QA, dont done est enregistré au front exécutant l'état final, QB signale la fin dès l'entrée dans cet état et économise une période de protocole. Les périodes des bancs étant différentes (20 ns pour QA et 10 ns pour QB), les durées en ns ne permettent pas de conclure à un gain de fréquence sur FPGA. La comparaison des temps à fréquence maximale et du nombre de blocs logiques nécessite la synthèse et l'implémentation sur carte demandées en question c).